

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I _Narmatha J (192521341)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _Narmatha J_

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

ANSWERS:

1. HDFS-Based Storage Solution for a Media Platform

Problem:

A media platform stores large amounts of videos, images, audio, and user-generated content. The system must maintain more than 99.9% availability, use a maximum replication factor of 3, and keep storage costs low.

Proposed Solution:

- Use HDFS as the main distributed storage system.
- Divide large media files into HDFS blocks and distribute them across different DataNodes.
- Set the replication factor to 3. This provides fault tolerance without exceeding the given constraint.
- Use NameNode High Availability with an Active and Standby NameNode.
- Store replicas on different DataNodes/racks so that failure of one machine or rack does not make the data unavailable.
- Use erasure coding for older or less frequently accessed media to reduce storage overhead.
- Keep frequently accessed media with replication factor 3 and archive older media using cost-efficient storage.

Justification:

Replication factor 3 provides protection against DataNode failures while satisfying the maximum replication constraint. NameNode HA removes the single point of failure. Erasure coding can reduce the storage overhead for cold data. Therefore, the design provides high availability while controlling storage cost.

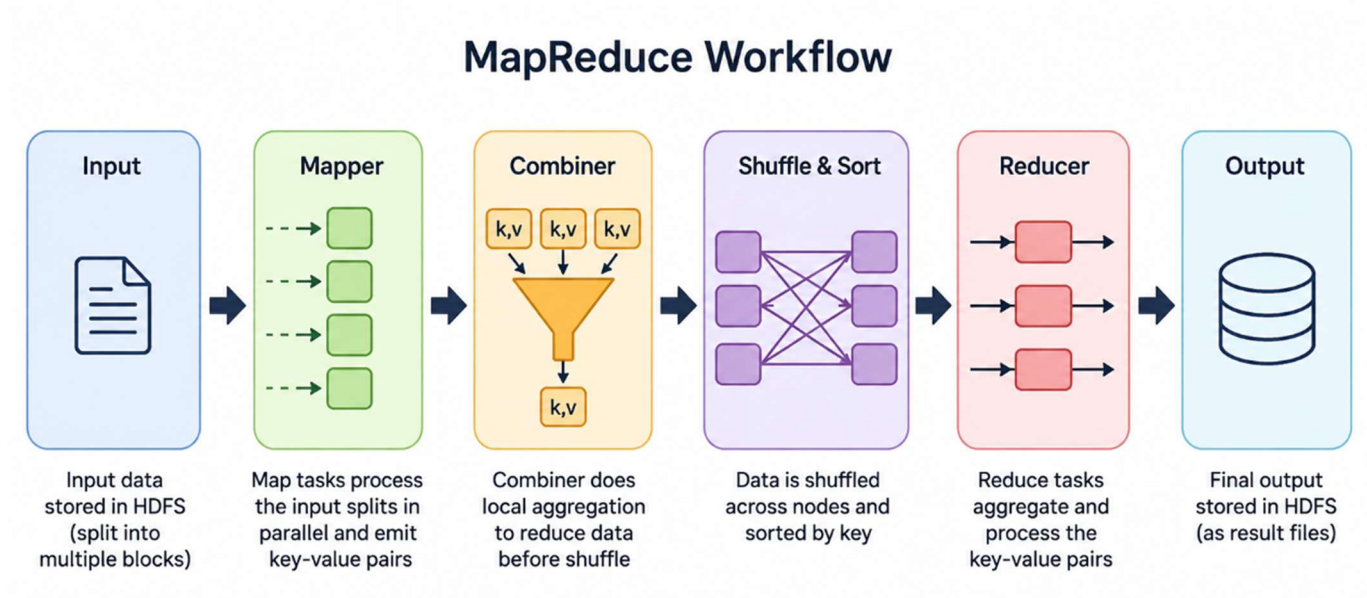
2. MapReduce Workflow for Processing Logs Within 20 Minutes

Problem:

A large amount of log data must be processed within a 20-minute batch window, but the Hadoop cluster has a limited number of nodes.

Proposed Job Design:

Use a MapReduce job with the following stages:



- Store incoming logs in HDFS.
- Divide the logs into blocks so that multiple mappers can process them in parallel.
- Each Mapper reads log records and extracts useful information such as IP address, status code, URL, or error type.
- Use a Combiner to perform local aggregation before data is transferred to reducers.
- Shuffle and Sort groups similar keys together.
- Reducers perform final aggregation and store the result in HDFS.

Task Distribution:

For example, if the cluster has 5 worker nodes:

- Distribute input blocks across all available nodes.
- Run multiple mapper tasks on each node depending on CPU and memory.
- Use fewer reducers than mapper tasks to avoid excessive resource usage.
- Give priority to data-local tasks to reduce network transfer.
- Monitor task progress and restart failed tasks automatically.

Constraint Handling:

The Combiner reduces network traffic, while parallel mappers reduce processing time. Appropriate partitioning and task distribution prevent overloaded nodes. If the processing time approaches 20 minutes, additional mapper tasks can be allocated within the available resources.

Result:

The workflow can process the log batch within the required 20-minute window while efficiently using the limited cluster nodes.

3. YARN Scheduler Policy for ETL, Reporting and ML Jobs

Problem:

A YARN cluster must handle three types of workloads:

- ETL jobs
- Reporting jobs
- Machine Learning jobs

The scheduler must provide fairness, give priority to ETL, and recover from node failures.

Proposed Scheduler Policy:

Use YARN Capacity Scheduler with separate queues:

Queue	Priority	Purpose
ETL	High	Data ingestion and transformation
Reporting	Medium	Business reports and queries
ML	Normal	Machine learning and analytics

Policy:

1. Give the ETL queue higher priority because ETL prepares data required by other workloads.
2. Reserve a minimum amount of cluster resources for reporting and ML so they are not completely blocked.
3. Allow unused resources from one queue to be temporarily used by another queue.
4. Apply fair resource allocation within each queue.
5. Set maximum resource limits so one workload cannot consume the entire cluster.

6. Enable YARN's automatic recovery mechanisms for failed NodeManagers.
7. If a node fails, running tasks are detected and rescheduled on healthy nodes.

Justification:

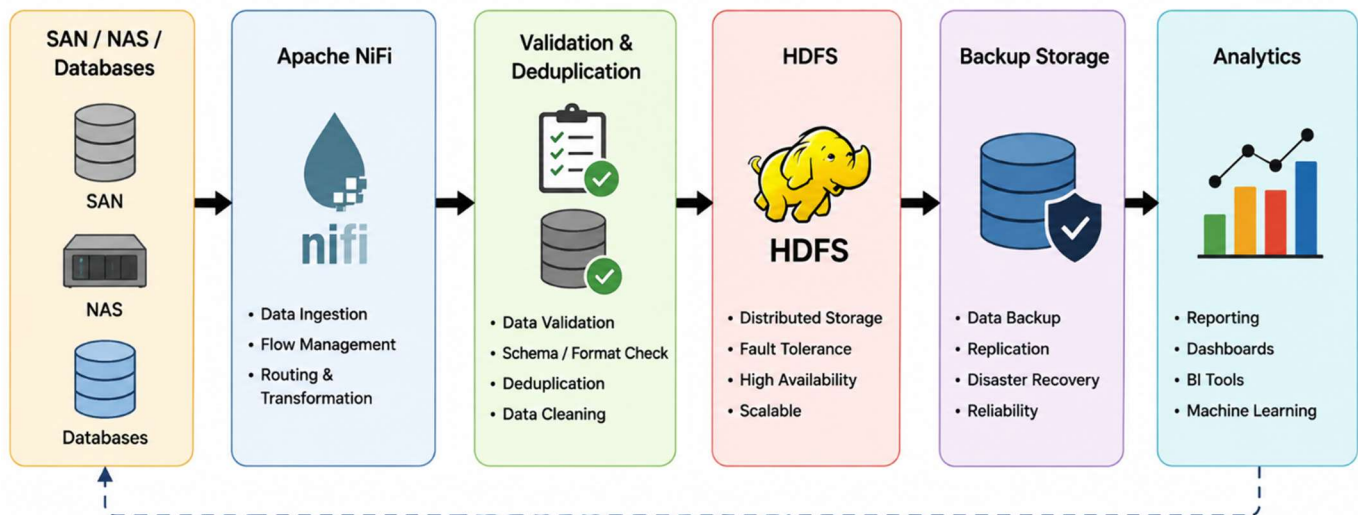
This design satisfies all constraints. ETL receives priority, while minimum resources ensure fairness for reporting and ML. Dynamic resource sharing improves cluster utilization. Automatic task rescheduling allows jobs to continue after node failures.

4. Enterprise Data Ingestion from SAN, NAS and Databases

Problem:

An enterprise needs to collect data from SAN, NAS, and operational databases while maintaining reliable backups and avoiding unnecessary duplication.

Proposed Architecture:



Components:

- Apache NiFi: Collects and routes data from different sources.
- Database connectors: Extract data from operational databases using JDBC.

- SAN/NAS connectors: Transfer files from enterprise storage systems.
- Data validation: Checks file format, schema, timestamps, and corrupted records.
- Deduplication: Uses file hashes, record IDs, or timestamps to identify duplicate data.
- HDFS: Stores the cleaned and consolidated data.
- Backup: Important data is copied to a secondary storage system or backup cluster.

Constraint Handling:

- NiFi provides controlled and traceable data movement.
- Deduplication prevents unnecessary copies from consuming storage.
- HDFS provides distributed storage and fault tolerance.
- Backup storage protects against major system failures.
- Metadata and checksums can be used to verify that transferred data is complete and unchanged.

Result:

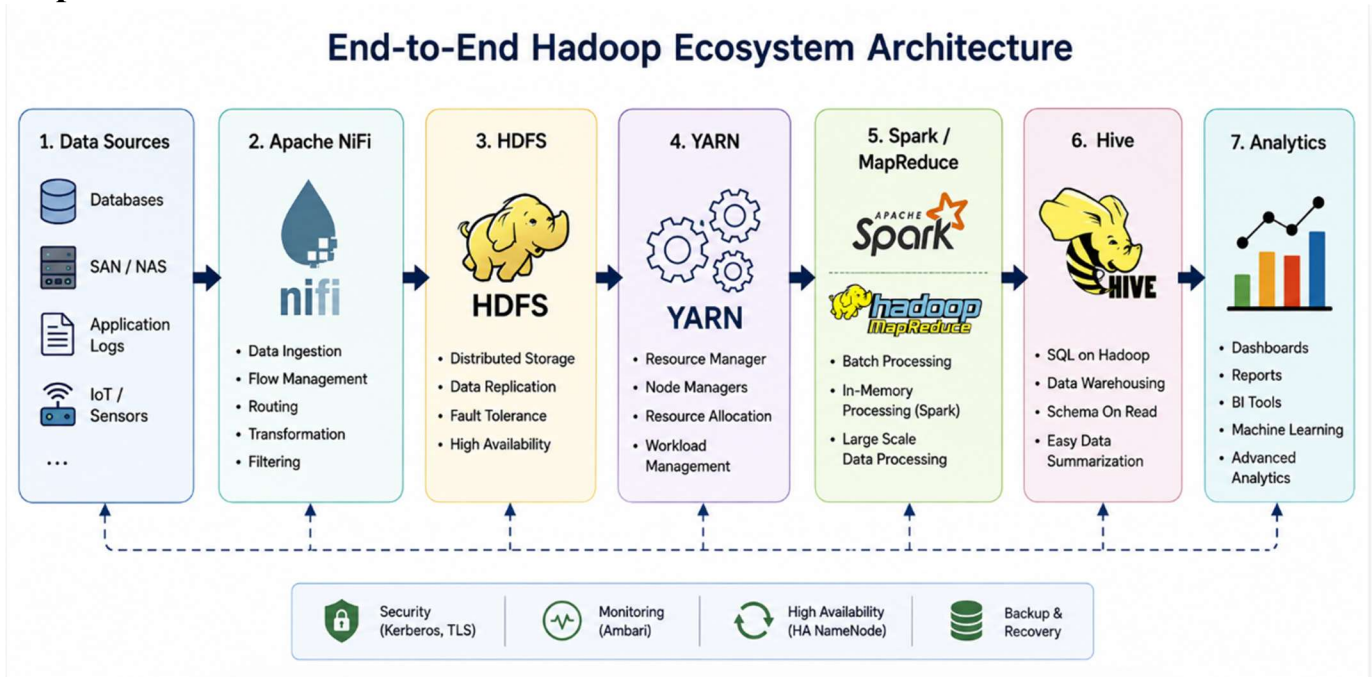
The architecture provides reliable ingestion from all three sources while reducing duplicate storage and maintaining backup protection.

5. End-to-End Hadoop Ecosystem Solution

Problem:

An organization needs a Hadoop-based system that supports secure data movement, high availability, and integration with Apache NiFi or ETL pipelines.

Proposed Architecture:



Design:

1. Data Sources

- Databases
- SAN/NAS
- Application logs
- IoT or transaction data

2. Apache NiFi

- Collects data from different sources.
- Controls data flow.
- Performs filtering, routing, and basic transformation.

3. Secure Data Movement

- Use encryption such as TLS during data transfer.
- Use authentication and access controls.
- Protect sensitive data using Hadoop security mechanisms such as Kerberos.

4. HDFS

- Stores large datasets across multiple DataNodes.
- Use replication and separate failure domains for fault tolerance.
- Configure NameNode High Availability.

5. YARN

- Manages CPU and memory resources.
- Runs MapReduce, Spark, and other processing jobs.
- Allocates resources according to workload priority.

6. Processing

- Use MapReduce for batch processing.
- Use Spark for faster analytics and machine-learning workloads.
- Use Hive for SQL-based analysis.

7. Backup and Recovery

- Maintain replicated data.
- Use secondary/backup storage for critical datasets.
- Automatically restart failed tasks on healthy nodes.

How Constraints Are Satisfied:

Constraint	Solution
Secure data movement	NiFi + TLS + authentication
High availability	HDFS replication + NameNode HA
ETL integration	Apache NiFi + ETL pipelines
Resource management	YARN
Large-scale processing	MapReduce/Spark
Data analysis	Hive
Failure recovery	Replication + task rescheduling

Conclusion:

The proposed Hadoop ecosystem provides a secure and highly available platform while allowing NiFi/ETL pipelines to efficiently move data into HDFS. YARN manages resources, while Spark, MapReduce, and Hive provide processing and analytics capabilities.